Deep Learning with Applications in Natural Language Processing

Lecture 2. Language models. From n-grams to neural networks

Mihaela Breabăn · Mădălina Răschip · Diana Trandabăț

Faculty of Computer Science, Alexandru Ioan Cuza University of Iași
Academic year 2026–2027

Today

1. What a language model is and what it is used for
2. N-gram models, maximum likelihood, log probabilities
3. Generating text: greedy, sampling, temperature
4. Zipf's law and data sparsity
5. Evaluation: entropy and perplexity
6. Smoothing, unknown words and skip-grams
7. Subword tokenization: BPE and relatives
8. Neural language models, word2vec skip-gram, LLMs

What is a language model?

Given a context, predict a probability distribution over what comes next.

context → language model → P(next token | context)

Equivalent view: assign a probability to a complete token sequence, $P(w_1 \ldots w_n)$.

The chain rule, later today, connects the two views.

Why model the probability of a sentence?

Application	A good model prefers…
Speech recognition	“I ate a cherry” over “Eye eight uh Jerry”
OCR, handwriting	the more probable reading
Machine translation	the more fluent translation
Spelling correction	“Their are problem wit this sentences.”
Generation	the more natural continuation

Predicting the next word

Please turn off your cell _____
Your program does not _____

A language model also predicts how a sentence continues: predictive keyboards, autocomplete and, at scale, today's LLMs.

Shannon's game

Guess the next letter

W → Wh → Wha → What d …

Guess the next word

What → What do → What do you …

Humans predict well from context. A language model learns to do the same from data.

From one guess to a distribution

Context: *Please turn off your cell …*

Candidate	Probability
phone	0.62
device	0.11
screen	0.07
… every other token	0.20 in total

The probabilities over all possible next tokens sum to 1. Illustrative values.

Real-word spelling errors

They are leaving in about fifteen **minuets**.

The study was conducted mainly **be** John Black.

Can they **lave** him my messages?

Hopefully, all **with** continue smoothly.

I need to **notified** the bank.

He is trying to **fine** out.

Every word exists in the dictionary. Only context, i.e. a language model, reveals the error.

Terminology

Term	Meaning
Sentence	unit of written language
Utterance	unit of spoken language
Word form	inflected form as it appears
Lemma	shared base form (same stem, POS, sense)
Types	distinct words: vocabulary size $\|V\|$
Tokens	total number of running words

Example: types and tokens

> The cat saw the other cat .

Count	Value	Which
Tokens	7	The, cat, saw, the, other, cat, .
Types (lowercased)	5	the, cat, saw, other, .

Whether “The” and “the” are one type is a normalization decision (Lecture 1).

What carries over from Lecture 1?

raw text → tokens → types / vocabulary V → counts

- A tokenization decision changes the sequence the LM sees.
- A type is a distinct token value; $|V|$ is the vocabulary size.
- Normalization can merge or split counts before the LM is ever trained.

From equal probabilities to frequencies

0-gram

Every word equally likely: $P(w) = 1/|V|$. With 100,000 words, 0.00001 each.

Unigram

$P(S) = P(w_1) \times P(w_2) \times \ldots \times P(w_n)$. Frequent words are more likely; context ignored.

N-gram

$P(S) = P(w_1) \times P(w_2|w_1) \times \ldots$ The previous words matter.

Joint and conditional probability

Joint

P(A, B): the probability that A and B occur together.

P(cell, phone): the bigram *cell phone*

Conditional

P(B | A): the probability of B when A is already known.

P(phone | cell): *phone*, given that we saw *cell*

$$P(A, B) = P(A) \times P(B \mid A)$$

The chain rule

$$P(w_1, w_2) = P(w_1) \cdot P(w_2 \mid w_1)$$
$$P(w_1 \ldots w_n) = P(w_1)\, P(w_2|w_1)\, P(w_3|w_1,w_2) \ldots P(w_n|w_1 \ldots w_{n-1})$$

$$P(\text{the dog barks}) = P(\text{the}) \cdot P(\text{dog} \mid \text{the}) \cdot P(\text{barks} \mid \text{the dog})$$

The exact context grows with the sentence

Please turn off your cell phone

Step	Factor of the chain rule
1	P(please)
2	P(turn | please)
3	P(off | please turn)
…	…
6	P(phone | please turn off your cell)

The last factor has a five-word history. A separate probability for every possible history cannot be estimated from finite data.

Why conditional probabilities?

- With 10,000 types, uniform P(w) = 1/10,000 ignores that *the* is far more frequent than *dog*
- Relative frequencies fix that, but still ignore context
- *dog* may be rare overall, yet P(dog | barking) is high, i.e. *dog* is frequent after *barking*.

N-gram models and the Markov assumption

Estimating P(phone | Please turn off your cell) needs too many parameters. An N-gram model keeps only N−1 words of context.

Model	Example
Unigram	P(phone)
Bigram	P(phone | cell)
Trigram	P(phone | your cell)

An N-gram model is a Markov model of order N−1.

Example: the Markov assumption

P(phone | please turn off your cell)	
≈ P(phone | your cell)	trigram
≈ P(phone | cell)	bigram

We need counts only for short sequences such as “cell phone”, which a corpus actually contains.

Unigram, bigram, trigram

Unigram

$$P(w_t)$$

No context. A good baseline, weak prediction.

Bigram

$$P(w_t \mid w_{t-1})$$

One previous token. Easy to count, still sparse.

Trigram

$$P(w_t \mid w_{t-2}, w_{t-1})$$

Two previous tokens. More specific, much sparser.

More context gives more specific estimates, but fewer observations per context.

N-gram approximations

Chain rule:	$P(w_1^n) = \prod_k P(w_k \mid w_1^{k-1})$
Bigram:	$P(w_1^n) \approx \prod_k P(w_k \mid w_{k-1})$
N-gram:	$P(w_1^n) \approx \prod_k P(w_k \mid w_{k-n+1}^{k-1})$

Maximum likelihood estimation

Estimate probabilities from counts in a corpus:

Bigram:	$P(w_n \mid w_{n-1}) = C(w_{n-1}\ w_n) \,/\, C(w_{n-1})$
N-gram:	$P(w_n \mid \text{context}) = C(\text{context}\ w_n) \,/\, C(\text{context})$

Add **<s>** and **</s>** to every sentence and treat them as words, so the model is a proper distribution.

A bigram grammar fragment: “eat …”

Bigram	P
eat on	.16
eat some	.06
eat lunch	.06
eat dinner	.05
eat at	.04
eat a	.04
eat Indian	.04
eat today	.03

Bigram	P
eat Thai	.03
eat breakfast	.03
eat in	.02
eat Chinese	.02
eat Mexican	.02
eat tomorrow	.01
eat dessert	.007
eat British	.001

More bigrams from the same corpus

Bigram	P
<s> I	.25
<s> I'd	.06
<s> Tell	.04
<s> I'm	.02
I want	.32
I would	.29
I don't	.08

Bigram	P
I have	.04
want to	.65
want a	.05
want some	.04
want Thai	.01
to eat	.26
to have	.14

Bigram	P
to spend	.09
to be	.02
British food	.60
British restaurant	.15
British cuisine	.01
British lunch	.01

Computing a sentence probability

P(I want to eat British food)
= P(I|<s>) P(want|I) P(to|want)
  × P(eat|to) P(British|eat) P(food|British)
= .25 × .32 × .65 × .26 × .001 × .60 ≈ 0.0000081

P(I want to eat Chinese food) ≈ 0.00015 >> about 18 times more likely.

Counts capture syntactic preferences and corpus-specific usage patterns; “eat British” is rare in this restaurant-query corpus.

Computing in log space

Multiplying many small probabilities quickly underflows: 1,000 words with p = 0.001 give 10^{-3000}, below the smallest number a computer can store (about 10^{-308}).

So we add **log probabilities** instead:

$$\log P(\text{I want to eat British food})$$
$$= \ln .25 + \ln .32 + \ln .65 + \ln .26 + \ln .001 + \ln .60$$
$$= -1.39 - 1.14 - 0.43 - 1.35 - 6.91 - 0.51 = -11.72$$
$$\exp(-11.72) \approx 0.0000081$$

Example: Romanian bigrams from a real corpus

What follows **a**? (4,777 occurrences)

Next word	P(w | a)
fost	.084
se	.016
fi	.015
avut	.011
lui	.009

What follows **a fost**? (402 occurrences)

Next word	P(w | a fost)
de	.030
creat	.015
reînnoită	.012
consemnată	.012
un	.010

UD Romanian-RRT treebank, about 218,000 tokens, lowercased. The trigram distribution is already flat: even the top word has only 3%.

Exercise: build a bigram model

<s> there is a big house </s>
<s> i buy a house </s>
<s> they buy the new house </s>

1. Estimate all bigram probabilities with MLE.
2. Compute P(they buy a big house).
3. Compute P(they buy a new house).
4. What changes for *they buy a red house*?

Recall: $P(w_n \mid w_{n-1}) = C(w_{n-1}\, w_n) \,/\, C(w_{n-1})$

Exercise: the bigram counts

Context	Next	Count
<s>	there · / i · / they	1 · 1 · 1
there	is	1
is	a	1
a	big · house	1 · 1
big	house	1
i	buy	1

Context	Next	Count
buy	a · / the	1 · 1
they	buy	1
the	new	1
new	house	1
house	</s>	3

C(buy) = 2, C(buy a) = 1, so P(a | buy) = 1/2. C(house) = 3, C(house </s>) = 3, so P(</s> | house) = 1.

Exercise: the bigram model

P(there|<s>) = 1/3	
P(i|<s>)	= 1/3
P(they|<s>)	= 1/3
P(is|there)	= 1
P(a|is)	= 1
P(big|a)	= 1/2
P(house|a)	= 1/2
P(house|big)	= 1

P(buy|i)	= 1
P(buy|they)	= 1
P(a|buy)	= 1/2
P(the|buy)	= 1/2
P(new|the)	= 1
P(house|new)	= 1
P(</s>|house)	=1

Exercise: sentence probabilities

P(they buy a big house)
= P(they|<s>) P(buy|they) P(a|buy)
  × P(big|a) P(house|big) P(</s>|house)
= 1/3 × 1 × 1/2 × 1/2 × 1 × 1 = 1/12 ≈ 0.083

P(they buy a **new** house) = 0: every word is known, but “a new” never occurs in training, so P(new | a) = 0.

One unseen bigram makes the whole sentence impossible.

Unseen n-gram vs out-of-vocabulary word

Unseen n-gram: “a new”

Both words are in V; the pair was never observed.

Fix: smoothing, back-off, interpolation.

Unseen word: “red”

Add-one also works **if red is in V**, e.g. a vocabulary fixed in advance (|V| = 12):

$$P(\text{red} \mid a) = (0+1)/(2+12) = 1/14 \approx 0.071$$
$$P(\text{house} \mid \text{red}) = (0+1)/(0+12) = 1/12$$

Usually V is built from the training data, so red $\notin$ V and add-one spreads mass only over known words.

Adding test words to V afterwards uses the test set.
Standard fix: an <UNK> token in V, or subwords / bytes.

Generating text from a language model

Start with <s>, choose the next word from P(w | history), append it, repeat until </s>.

Greedy

Always take the most probable word. After “want” it is always “to”: fluent but repetitive.

Sampling

Draw a word at random, in proportion to its probability. Varied output, sometimes odd.

Top-k / top-p

Sample only among the k most probable words, or the smallest set covering probability p.

Temperature: sharpening or flattening the distribution

$$P_T(w) = \frac{P(w)^{1/T}}{\sum_{v \in V} P\,(v)^{1/T}}$$

Next word after “want”	T = 0.5	T = 1	T = 2
to	.990	.867	.606
a	.006	.067	.168
some	.004	.053	.150
Thai	<.001	.013	.075

Restaurant bigrams for “to, a, some, Thai”, renormalized.

T < 1 approaches greedy; T > 1 makes rare words more likely.

Problems of n-gram models

- **Sparse data:** most N-grams never occur in training
- **Domain change:** trained on the Wall Street Journal, tested on Shakespeare (see example below)
- **Larger N:** more fluent, but even sparser

Unigrams: Every enter now severally so, let
Bigrams: What means, sir. I confess she?
Trigrams: Sweet prince, Falstaff shall die.

Reliable estimates

- Smooth: never assign zero probability
- Back off: if the N-gram is unseen, use the (N−1)-gram

P(“And nothing but the truth”)	≈ 0.001
P(“And nuts sing on the roof”)	≈ 0

Zipf's law

Rank words by frequency f. Zipf (1949) observed that rank (r) × frequency (f) is roughly constant:

$$f \cdot r \approx k$$

$$p_r = \frac{f}{N} \approx \frac{A}{r}$$

N = total number of word occurrences.
A = proportionality constant

Example: Zipf's law in a 1-million-word corpus

Rank r	Predicted share A / r	Expected count
1	10%	100,000
10	1%	10,000
100	0.1%	1,000
10,000	0.001%	10

With A ≈ 0.1. In English, rank 1 is “the”.

Zipf's law: the long tail

Head

A few words (the, of, and) cover a large part of any text.

Long tail

Most word types are rare; many occur only once.

However large the corpus, most n-grams are seen rarely or never: this is **data sparsity**, and the reason we need smoothing and subwords.

Training and test corpora

- Train on a large corpus; evaluate on a **disjoint, held-out** test corpus
- Testing on training data gives an optimistic estimate
- Both should represent the target application
- Adapt a general model with a small, up-weighted in-domain corpus

Extrinsic vs intrinsic evaluation

Extrinsic (in vivo)

Measure the end application, e.g. ASR accuracy. Realistic, expensive.

Intrinsic

Measure how well the model predicts a test corpus. Cheap, less realistic.

Check at least once that intrinsic gains translate into extrinsic ones.

Entropy

$$H(X) = -\Sigma_x\, p(x) \log_2 p(x)$$

The expected number of bits needed to encode the next outcome. Comparing two models on the same corpus, the one with **lower cross-entropy** fits it better.

Example: entropy of simple sources

Source	Probabilities	Entropy
Fair coin	0.5, 0.5	1 bit
Biased coin	0.9, 0.1	0.47 bits
Fair die	1/6 each	2.58 bits

The more predictable the source, the lower the entropy. A good language model makes text more predictable.

Cross-entropy: average surprise on the correct next word

$$H = -\frac{1}{N}\sum_t \log_2 P(w_t \mid \text{context}_t) \qquad PP = 2^{H}$$

Model	Cross-entropy (bits per word)	Perplexity
A	2	$2^2 = 4$
B	3	$2^3 = 8$

Model A gives, on average, higher probability to the words that actually come next: it is as uncertain as choosing among 4 equally likely words, B among 8.

Perplexity

$$PP(W) = P(w_1\ w_2\ \ldots\ w_N)^{(-1/N)} = 2^{H(W)}$$

- The inverse probability of the test set, normalized by length
- 2 to the power of the cross-entropy: how many words the model hesitates between
- **Lower is better** predictive fit, on the same test set and vocabulary

Example: perplexity of a uniform model

A model that assigns 1/10 to each of the digits 0–9, tested on any string of N digits:

$P(w_1 \ldots w_N) = (1/10)\hat{}N$
$PP = ((1/10)\hat{}N)\hat{}(-1/N) = 10$

Perplexity 10: the model hesitates between 10 equally likely options at each step. The trigram model on WSJ (109) is like choosing among about 109 words.

Perplexity in practice

Trained on 38 million words of the Wall Street Journal (vocabulary 19,979), tested on 1.5 million other words:

	Unigram	Bigram	Trigram
Perplexity	962	170	109

Source: Jurafsky & Martin, Speech and Language Processing.

Lower perplexity does not automatically mean a better chatbot or system.

Speed also matters!

Caution: perplexity depends on the tokenization

Same sentence, same total probability: $\log_2 P(\text{sentence}) = -40$ bits. The sentence is 40 bytes long.

Tokenizer	Tokens	H (bits / token)	Perplexity
A	8	$40 / 8 = 5$	$2^5 = 32$
B	14	$40 / 14 \approx 2.86$	≈ 7.2

Per-token perplexities from different tokenizers are not comparable.
Compare with the same tokenizer, or normalize by a fixed unit, e.g. bits per byte: $40 / 40 = 1$ for both.

Why smoothing?

- Many rare but possible word sequences never occur in training
- MLE gives them probability zero, so the whole sentence gets zero (infinite perplexity)
- Smoothing moves some probability mass from seen to unseen events (**discounting**)

Unknown words

Words in the test set that never appeared in training are **out-of-vocabulary** (OOV).

- Add a special token <UNK> to the model
- Fix a vocabulary in advance; map all other words to <UNK>
- Or replace the first occurrence of each word in training with <UNK>

Subword tokenization (later today) largely removes the problem.

Example: mapping rare words to <UNK>

Vocabulary: the 10,000 most frequent training words

Test: the zorbling cat sat on the mat
Seen: the <UNK> cat sat on the mat

P(<UNK> | the) is estimated from all rare words seen after “the” in training.

Laplace (add-one) smoothing

Pretend every possible N-gram was seen once more than it was:

$$P(w_n \mid w_{n-1}) = (C(w_{n-1}\, w_n) + 1) \,/\, (C(w_{n-1}) + |V|)$$

- Simple, but gives too much mass to unseen events
- Add-k: add $0 < k < 1$ instead of 1

Exercise: Laplace smoothing

Same corpus. Possible next words: the 10 word types plus </s>, so |V| = 11.

P(they|<s>)	= (1+1)/(3+11) = 0.143
P(buy|they)	= (1+1)/(1+11) = 0.167
P(a|buy)	= (1+1)/(2+11) = 0.154
P(new|a)	= (0+1)/(2+11) = 0.077
P(house|new)	= (1+1)/(1+11) = 0.167
P(</s>|house) = (3+1)/(3+11) = 0.286	

Exercise: the smoothed result

P(they buy a new house)
$= 0.143 \times 0.167 \times 0.154 \times 0.077 \times 0.167 \times 0.286$
$\approx 1.3 \times 10^{-5}$

No longer zero. But *they buy a big house* drops from 0.083 to about 2.7×10^{-5}: add-one gives the same extra count to every continuation, even implausible ones, so it moves too much mass away from what was actually seen.

Kneser–Ney: the idea

I can't see without my reading _____

- “Kong” may be more frequent than “glasses” in the corpus
- But “Kong” almost only follows “Hong”; “glasses” follows many words
- Kneser–Ney backs off to how many **different contexts** a word appears in, so it prefers “glasses”

The standard choice for n-gram models. Other methods in the appendix.

Interpolation: combine specific and robust estimates

$$P(w \mid u\ v) = \lambda_3\, P_ML(w \mid u\ v) + \lambda_2\, P_ML(w \mid v) + \lambda_1\, P_ML(w)$$
$$\lambda_1 + \lambda_2 + \lambda_3 = 1$$

Trigram

specific, but often unseen

Bigram

less context, more data

Unigram

always available

Back-off uses the lower order only when the higher one is missing; interpolation always mixes them. The λ weights are tuned on the development set.

PART 2

Subword tokenization

Between words and characters: how modern language models build their vocabulary.

Why not words, why not characters?

Words

Huge vocabulary; unknown words become <UNK>; Zipf's tail is never learned.

Characters

Tiny vocabulary, no unknown words, but very long sequences.

Subwords

Frequent words stay whole; rare words split into known pieces.

Character-based tokenization

- Split text into characters; map each to its Unicode code point
- (+) Greatly reduces OOV if the character vocabulary covers the input alphabet; byte-level models guarantee coverage with 256 byte values
- (−) **Sequences** get very long, so attention and training cost more
- (−) A single character carries little meaning; the model must learn that happy, happiness and unhappiness share a root

Byte Pair Encoding (BPE)

1. Start with a vocabulary of single characters (plus an end-of-word mark)
2. Count all adjacent symbol pairs in the corpus
3. Merge the most frequent pair into a new symbol and **add** it to the vocabulary
4. Repeat for M merges, or until no pair repeats

The base characters stay in the vocabulary, so any new word can still be spelled. Vocabulary size = characters + M.

BPE example: the corpus

low_ new_ lowest_ newest_ widest_ lower_

Pair	Count	Pair	Count
l o	3	e s	3
o w	3	s t	3
w e	3	t _	3
w _	2	n e	2

11 base symbols: _ d e i l n o r s t w. Example after D. Gavriluț, EUROLAN 2023.

BPE example: the merges

Merge	Pair	Corpus after the merge
1	l + o	lo w _ · lo w e s t _ · lo w e r _
2	lo + w	low _ · low e s t _ · low e r _
3	e + s	low es t _ · n e w es t _ · w i d es t _
4	es + t	low est _ · n e w est _ · w i d est _
5	est + _	low est_ · n e w est_ · w i d est_

After 5 merges the vocabulary has 16 symbols; “est_” is a learned suffix.

Applying BPE to new text

Apply the learned merges, in the same order, to any word:

```
slowest → s l o w e s t _
        → s low est_
```

- No unknown words: the worst case is single characters
- Byte-level BPE (GPT-2 and later) starts from 256 bytes instead of characters

Other subword methods

Method	How the vocabulary is built
BPE	merge the most frequent pair
WordPiece (BERT)	likelihood-based subword vocabulary; used by BERT
Unigram LM	start big, prune pieces that matter least
SentencePiece	framework on raw text; supports BPE and Unigram

Tokenizer-free alternatives (bytes, learned chunks) and tokens for audio, images and sign language: Lecture 1.

Tokenization changes the language-modeling problem

More tokens

Longer sequences: more prediction steps, and less text fits in a fixed context window.

Different pieces

The model sees different reusable units and different frequency statistics.

A tokenizer is part of the model specification, not a neutral preprocessing detail.

PART 3

Neural language models

Generalizing beyond counts with learned word representations.

Why n-grams eventually fail

Problem	Example
Sparsity	most valid trigrams are never seen
Fixed context	a trigram remembers only two words
Domain shift	counts trained on news, tested on chat
No generalization	cat and dog share nothing
Parameter explosion	|V|^N possible histories

Smoothing, subwords and learned representations each answer part of this list.

Limits of n-grams: context

Usually N ≤ 4, but dependencies can be much longer:

> The **man** next to the large oak tree near the grocery store on the corner **is** tall.
> The **men** next to the large oak tree near the grocery store on the corner **are** tall.

The same holds for meaning: the bird … flies, the man … talks.

Limits of n-grams: no generalization

The cat is walking in the bedroom
A dog was running in a room

These should be about equally likely, because *cat/dog*, *the/a* and *room/bedroom* play similar roles. For an n-gram model they are unrelated symbols.

Sparse vs distributed representations

One-hot (sparse)

	v	h	s	r
▯	1	0	0	0
▭	0	1	0	0
◯	0	0	1	0
⬭	0	0	0	1

one dimension per shape

Distributed (dense)

	vert.	horiz.	squared	round
▯	1	0	1	0
▭	0	1	1	0
◯	1	0	0	1
⬭	0	1	0	1

shared features

The same problem for words

One-hot

cat	[0 0 1 0 … 0]
dog	[0 1 0 0 … 0]

|V| dimensions; every pair of words equally far apart.

Embeddings

cat	[0.7 0.1 −0.3 …]
dog	[0.6 0.2 −0.2 …]

m ≪ |V| dimensions; similar words close together.

Check out this Word Embedding Demo

A neural probabilistic language model

- Give every word a learned **vector** (embedding) of $m \ll |V|$ numbers
- Predict the next word from the vectors of the previous $n-1$ words
- Learn the vectors and the predictor **together**

What is learned about *cat* now transfers to *dog*, because their vectors end up close.

Bengio et al. (2003), JMLR 3: 1137–1155.

Architecture

Previous n−1 words

indices

→

Lookup in C

concatenate embeddings

→

Hidden layer(s)

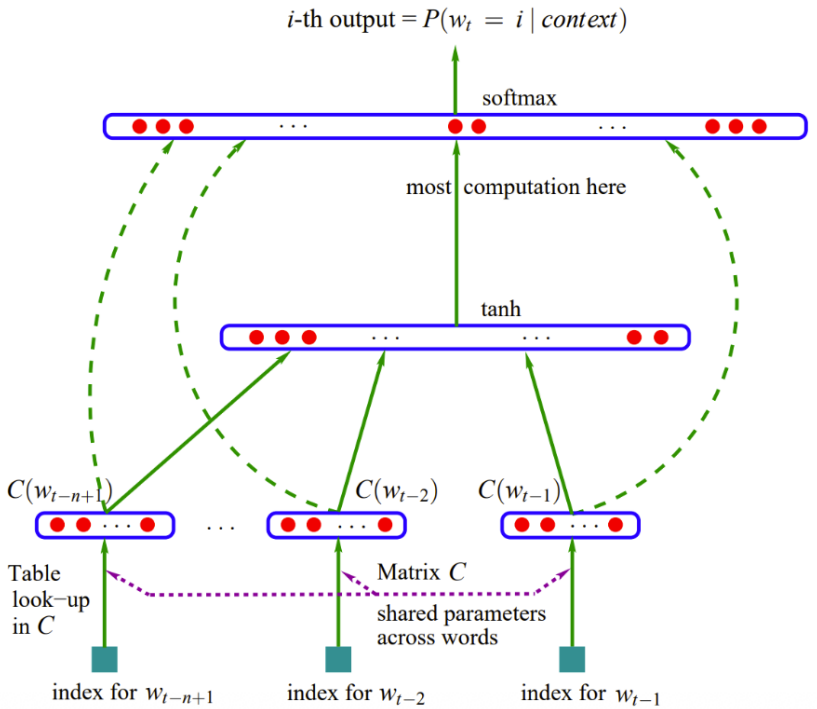
tanh

→

Softmax

$P(w_t = i \mid \text{context})$

Most of the computation is in the output layer: one score for every word in V.



Neural language model representation

Bengio, Yoshua, et al. "A neural probabilistic language model." The journal of machine learning research 3 (2003): 1137-1155.

From scores to probabilities: softmax

Next word	Score (logit - z)	Softmax
house	2.0	0.67
room	1.0	0.24
car	0.0	0.09

$P(i) = \exp(z_i) / \Sigma_j \exp(z_j)$

Positive values that sum to 1, in the same order as the scores.

Training raises the probability of the word that actually came next, by minimizing cross-entropy.

Temperature can also be added:

$$P_T(i) = \frac{\exp(z_i/T)}{\sum_j \exp(z_j/T)}$$

LLMs are language models too

- Same task: P(next token | previous tokens), trained by maximizing likelihood
- Same evaluation: cross-entropy and perplexity on held-out text
- Same generation: one token at a time, with sampling and temperature
- Different: the history is summarized by a neural network over thousands of tokens, not by counts over N−1 words

Perplexities are per token, so they are comparable only with the same tokenizer. Bits per byte normalizes by text length instead (used e.g. for BLT).

Not every pretrained language model predicts the next word

Autoregressive (GPT-style)

Predict the next token from the previous ones: $P(w_1 \ldots w_n) = \prod P(w_t \mid w_{<t})$.

Natural for generation.

Masked (BERT-style)

Hide some tokens and predict them from the left and right context. Good for representations, but not a left-to-right model of the sequence.

Both objectives return in the lectures on pretraining and Transformers.

N-gram models are still in use

Filtering pretraining data

CCNet scores Common Crawl paragraphs with a 5-gram Kneser–Ney model (KenLM) trained on Wikipedia. Low perplexity = close to Wikipedia-like text; data is split into head, middle and tail.

∞-gram at trillion scale

Infini-gram (infini-gram.io/demo) counts n-grams of any length over 5 trillion tokens with suffix arrays. It predicts the next token correctly 47% of the time and lowers neural LMs' perplexity when combined with them.

Wenzek et al. (2020), LREC · Liu et al. (2024), COLM.

What changes from n-grams to neural LMs?

	N-gram LM	Neural LM
Predicts	next-word distribution	next-word distribution
Context	fixed, short	encoded by a learned function
Estimator	normalized counts	neural network + softmax
Word representation	symbol / id	learned vector
Generalization	shared local n-grams	shared parameters and vectors

The objective stays the same; the representation and the estimator change.

Where neural language models went next

Step	Idea
Word embeddings	learn word vectors on their own (word2vec, GloVe)
Recurrent LMs	variable-length context summarized in a hidden state
Attention	look back directly at any earlier position
Transformers	self-attention + feed-forward blocks, pretrained at scale

Each is covered in the following lectures.

The prediction problem has not changed.
The way we represent context has.

Summary

- A language model predicts the next token; the chain rule turns this into sentence probabilities, computed in log space
- N-grams + MLE are easy to train but suffer from sparsity (Zipf)
- Generation samples one token at a time; temperature controls how adventurous it is
- Smoothing, back-off and skip-grams fight sparsity; perplexity measures predictive fit
- Subwords remove most unknown words; neural models and word2vec learn word vectors
- LLMs solve the same prediction problem with a far richer view of context

Bridge to Lecture 3: structure

A language model tells us which sequences are probable. But the same words can carry different structures and meanings:

> I saw the lady with the binoculars.

Who has the binoculars? **Lecture 3:** constituency, dependency and parsing, where probabilities return, this time over trees.

Try it: Google Books Ngram Viewer

books.google.com/ngrams plots how often words and phrases appear in books over time.

- Compare two spellings or synonyms
- Find when a term entered the language
- Discuss: what does this corpus represent, and what not?

References (1/4): n-gram models

Jurafsky & Martin (2026). Speech and Language Processing, 3rd ed., online manuscript, Aug. 19, 2026
Shannon (1951). Prediction and entropy of printed English. Bell System Technical Journal 30(1): 50–64
Zipf (1949). Human Behavior and the Principle of Least Effort. Addison-Wesley Press
Kneser & Ney (1995). Improved backing-off for M-gram language modeling. ICASSP, 181–184
Chen & Goodman (1999). An empirical study of smoothing techniques for language modeling. Computer Speech & Language 13(4)

References (2/4): subword tokenization

Sennrich, Haddow & Birch (2016). Neural Machine Translation of Rare Words with Subword Units. ACL, 1715–1725 (ACL 2026 Test-of-Time Award)
Wu et al. (2016). Google's Neural Machine Translation System: Bridging the Gap between Human and Machine Translation. arXiv:1609.08144
Kudo (2018). Subword Regularization: Improving Neural Network Translation Models with Multiple Subword Candidates. ACL, 66–75
Kudo & Richardson (2018). SentencePiece: A simple and language independent subword tokenizer and detokenizer for Neural Text Processing. EMNLP (demo), 66–71

References (3/4): neural models and generation

Bengio, Ducharme, Vincent & Jauvin (2003). A Neural Probabilistic Language Model. JMLR 3: 1137–1155
Mikolov, Chen, Corrado & Dean (2013). Efficient Estimation of Word Representations in Vector Space. arXiv:1301.3781
Mikolov, Sutskever, Chen, Corrado & Dean (2013). Distributed Representations of Words and Phrases and their Compositionality. NeurIPS
Holtzman, Buys, Du, Forbes & Choi (2020). The Curious Case of Neural Text Degeneration. ICLR

References (4/4): recent work and data

Wenzek et al. (2020). CCNet: Extracting High Quality Monolingual Datasets from Web Crawl Data. LREC
Liu, Min, Zettlemoyer, Choi & Hajishirzi (2024). Infini-gram: Scaling Unbounded n-gram Language Models to a Trillion Tokens. COLM
Universal Dependencies. Romanian-RRT treebank. universaldependencies.org/treebanks/ro_rrt/

Thank you!

diana.trandabat@info.uaic.ro

PART A

Appendix

Derivations, extra methods and background for reference.

Skip-grams: n-grams with gaps

A **k-skip-n-gram** is a sequence of n words in their original order, with at most k words skipped in total.

Why

Sparsity again: an exact n-gram may be unseen, while the same words with a gap were seen. Skip-grams collect more evidence from the same corpus.

Cost

Many more pairs to count, and some are unrelated words that only happen to be close.

Guthrie et al. (2006) measured how many test n-grams are covered when training counts include skips.

Example: skip-bigrams

the AI Act was published

Type	Pairs	Count
bigrams (0 skips)	the AI · AI Act · Act was · was published	4
+ 1-skip	the Act · AI was · Act published	7
+ 2-skip	the was · AI published	9

A sentence of m words has m − 1 bigrams; allowing skips adds pairs that are further apart.

Appendix: Zipf's law, predicting frequencies

- A word seen n times has rank $r_n \approx AN / n$
- Words seen exactly n times: $I_n = r_n - r_{n+1} = AN / (n(n+1))$
- Fraction of the vocabulary seen exactly n times: $1 / (n(n+1))$

So, in theory, **half** of all word types occur only once.

Appendix: other smoothing methods

Method	Idea
Absolute discounting	subtract a constant from every count
Witten–Bell	use types seen once to estimate unseen ones
Good–Turing	re-estimate rare counts from counts of counts
Kneser–Ney	weigh by the number of contexts a word appears in
Interpolation	mix N-gram, (N−1)-gram, … estimates

Modified Kneser–Ney is the standard best choice for n-gram models.

Appendix: the embedding matrix C

- C maps every word i in V to a vector $C(i) \in \mathbb{R}^m$
- C is a $|V| \times m$ matrix of free parameters, shared across positions
- m is small (30 to 100 in the original experiments) compared to |V| (over 15,000 words)